

Cerebro SDK: Copy-Paste Code Guide

Real Methods, Real Examples, Real Results

Writer Security Engineering

November 6, 2025

How To Use This Guide

Every code example follows this pattern:

1. **Use Case:** What you want to accomplish
2. **Code:** Exact method calls with parameters
3. **Result:** What you get back
4. **Notes:** Common gotchas and tips

All examples are production-tested and ready to copy-paste.

Python: 30-Second Setup

Install:

```
uv pip install -e /path/to/cerebro  
# Or add to pyproject.toml dependencies
```

Get a database session:

```
from cerebro.core.database import async_session_factory  
  
async with async_session_factory() as db:  
    # Use SDK here  
    pass
```

Import what you need:

```
from cerebro_sdk import (  
    AuthSession,      # Login and tokens  
    AgentManager,     # Agent sessions  
    FindingService,   # Security findings  
    AgentReviewManager, # Review queue  
)
```

TypeScript: 30-Second Setup

Install:

```
npm install @writerai/cerebro-sdk
```

Initialize client:

```
import { CerebroSDK } from "@writerai/cerebro-sdk";

const sdk = new CerebroSDK({
  baseUrl: "https://cerebro.writer.com",
  async getAccessToken() {
    return await yourAuthService.getToken();
  }
});
```

Use the clients:

```
// All functionality under sdk.{module}
sdk.agents.createSession(...)
sdk.findings.list(...)
sdk.securityCenter.listVendors(...)
```

Create an Agent Session

Use Case: Start a new agent conversation

Python:

```
from cerebro_sdk import AgentManager

agents = AgentManager(db)
session = await agents.create_session(
    org_id=UUID("your-org-id"),
    agent_type="security_analyst", # or incident_responder
    created_by="your-email@writer.com",
    context={"finding_ids": ["finding-123"]},
    title="Investigate critical finding"
)

# Result: AgentSessionRecord
print(session.session_id) # UUID of new session
print(session.is_active) # True
```

TypeScript:

```
const session = await sdk.agents.createSession({
  orgId: "your-org-id",
  agentType: "security_analyst",
  context: { findingIds: ["finding-123"] }
});
```

Send a Message to Agent

Use Case: Ask the agent a question

Python:

```
message = await agents.add_message(  
    session_id=session.session_id,  
    role="user", # or "assistant" or "system"  
    content={"text": "What is the blast radius of this finding?"}  
)
```

```
# Result: AgentMessageRecord  
print(message.message_id)  
print(message.role)  
print(message.created_at)
```

TypeScript - With Streaming:

```
const stream = sdk.agents.streamSession({  
    sessionId: session.sessionId,  
    message: "What is the blast radius?"  
});
```

```
// Collect all events  
import { collectAgentStream } from "@writerai/cerebro-sdk";  
const result = await collectAgentStream(stream);
```

List All Sessions for an Org

Use Case: See recent agent activity

Python:

```
sessions, total = await agents.list_sessions(  
    org_id=org_id,  
    agent_type="security_analyst",  # Optional filter  
    limit=50,  
    offset=0  
)
```

```
for session in sessions:  
    print(f"{session.session_id}: {session.title}")  
    print(f"  Created: {session.created_at}")  
    print(f"  Active: {session.is_active}")  
    print(f"  Context: {session.context}")
```

TypeScript:

```
const { sessions } = await sdk.agents.listSessions({  
    agentType: "security_analyst",  
    limit: 50  
});
```

```
sessions.forEach(s => {  
    console.log(`${s.sessionId}: ${s.title}`);
```

Get Session With Full Message History

Use Case: View complete conversation

Python:

```
# Get session metadata
session = await agents.get_session(session_id)

# Get all messages in the session
messages = await agents.list_messages(
    session_id=session_id,
    limit=100
)

for msg in messages:
    print(f"{msg.role}: {msg.content.get('text', '')}")
```

TypeScript:

```
// Get session with messages in one call
const sessionDetail = await sdk.agents.getSession(
    sessionId,
    { messageLimit: 100 }
);

console.log(sessionDetail.session.title);
sessionDetail.messages.forEach(msg => {
```


Link Findings to a Session

Use Case: Add more findings to existing investigation

Python:

```
updated_session = await agents.link_findings(  
    session_id=session_id,  
    finding_ids=["finding-456", "finding-789"],  
    regenerate=True # Re-run finding generation  
)
```

```
# Now session.context includes all finding IDs  
print(updated_session.context["finding_ids"])  
# ['finding-123', 'finding-456', 'finding-789']
```

Use Case: Create session for specific incident

```
incident_session = await agents.create_incident_session(  
    org_id=org_id,  
    created_by="oncall@writer.com",  
    incident_id="INC-2025-001",  
    agent_type="incident_responder"  
)
```

Find Sessions by Finding or Incident

Use Case: "Show me all agent sessions for this finding"

Python:

Find sessions that investigated a specific finding

```
sessions = await agents.sessions_for_finding(  
    org_id=org_id,  
    finding_id="finding-123",  
    limit=20  
)
```

Find sessions for an incident

```
incident_sessions = await agents.sessions_for_incident(  
    org_id=org_id,  
    incident_id="INC-2025-001",  
    limit=20  
)
```

```
for sess in sessions:  
    print(f"Session {sess.session_id} investigated this finding")
```

Notes:

- ▶ Uses PostgreSQL JSON containment for fast lookups
- ▶ Returns most recent sessions first

Get Agent Memory Statistics

Use Case: Understand what the agent remembers

Python:

```
stats = await agents.get_memory_stats(  
    session_id=session_id,  
    since_hours=24,           # Optional: recent memories only  
    scope_type="finding",     # Optional: filter by scope  
    role="assistant"         # Optional: filter by role  
)  
  
if stats:  
    print(f"Total memories: {stats.total_entries}")  
    print(f"Recent (24h): {stats.recent_entries}")  
    print(f"Avg decay score: {stats.average_decay}")  
    print(f"Total tokens: {stats.token_total}")  
  
    print("\nRole distribution:", stats.role_distribution)  
    # {'user': 10, 'assistant': 15, 'system': 3}  
  
    print("\nScope distribution:", stats.scope_distribution)  
    # {'finding': 8, 'incident': 2, 'asset': 5}  
  
    print("\nTop 5 memories by decay score:")  
    for mem in stats.top_memories:
```

Get Review Queue Summary

Use Case: Dashboard showing pending work

TypeScript:

```
const summary = await sdk.agents.getReviewQueueSummary();

console.log(`Generated at: ${summary.generatedAt}`);

summary.statusCounts.forEach(stat => {
  console.log(`${stat.status}: ${stat.count} tasks`);
});
// pending: 15, in_progress: 5, completed: 30

console.log(`\nPending summary:`);
console.log(`  Total: ${summary.pending.totalPending}`);
console.log(`  Avg age: ${summary.pending.avgAgeHours}h`);
console.log(`  Oldest: ${summary.pending.oldestTaskAge}h`);

summary.priorityBreakdown.forEach(p => {
  console.log(`${p.priority}: ${p.count}`);
});
// high: 5, medium: 8, low: 2
```

List Pending Review Tasks

Use Case: Get all tasks that need attention

Python:

```
from cerebro_sdk import AgentReviewManager

review = AgentReviewManager(db)

tasks = await review.list_tasks(
    org_id=org_id,
    status="pending", # or "in_progress", "completed"
    limit=50
)

for task in tasks:
    print(f"Task {task.task_id}: {task.title}")
    print(f"  Status: {task.status}")
    print(f"  Priority: {task.priority}")
    print(f"  Created: {task.created_at}")
    print(f"  Assigned to: {task.assigned_to}")
    print(f"  Session: {task.session_id}")
```

TypeScript - With Pagination:

```
const page = await sdk.agents.listReviewTasksPage({
  status: "pending",
```

Bulk Update Review Tasks

Use Case: Assign multiple tasks at once

TypeScript:

```
const updated = await sdk.agents.bulkUpdateReviewTasks({
  taskIds: ["task-1", "task-2", "task-3"],
  status: "in_progress",
  assignedTo: "analyst@writer.com",
  priority: "high",
  dueAt: new Date(Date.now() + 24 * 60 * 60 * 1000) // 24h from now
});
```

```
console.log(`Updated ${updated.length} tasks`);
```

Python:

```
from datetime import datetime, timedelta
```

```
updated = await review.bulk_update_tasks(
    task_ids=[task1.task_id, task2.task_id],
    status="in_progress",
    assigned_to="analyst@writer.com",
    priority="high",
    due_at=datetime.now() + timedelta(hours=24)
)
```

Check SLA Status

Use Case: Find tasks about to breach SLA

TypeScript:

```
// Get tasks at risk
const atRisk = await sdk.agents.listReviewTasksSlaAtRisk({
  thresholdHours: 24 // Tasks with < 24h remaining
});

atRisk.forEach(task => {
  console.log(`${task.taskId}: ${task.title}`);
  console.log(`  Time remaining: ${task.remainingHours}h`);
});

// Get full SLA summary
const slaSummary = await sdk.agents.getReviewTasksSlaSummary();

console.log(`Breached: ${slaSummary.breached}`);
console.log(`At risk: ${slaSummary.atRisk}`);
console.log(`On track: ${slaSummary.onTrack}`);
console.log(`Compliance: ${slaSummary.complianceRate}%`);
```

Python:

```
at_risk = await review.list_tasks_sla_at_risk(
    org_id=org_id,
```

Add Comments and Track History

Use Case: Document review decisions

Python:

Add comment

```
comment = await review.add_comment(  
    task_id=task.task_id,  
    author="analyst@writer.com",  
    content="Investigated - false positive, safe to close",  
    metadata={"severity_reassessment": "low"}  
)
```

Get all comments

```
comments = await review.list_comments(task_id=task.task_id)
```

for c in comments:

```
    print(f"{c.author} at {c.created_at}:")  
    print(f"    {c.content}")
```

Get audit history

```
history = await review.get_task_history(task_id=task.task_id)
```

for h in history:

```
    print(f"{h.changed_by} changed {h.field_name}:")  
    print(f"    {h.old_value} -> {h.new_value}")
```


Export Review Tasks With Context

Use Case: Sync to external ticketing system

Python:

```
exports = await review.export_tasks(  
    org_id=org_id,  
    status="completed", # Optional filter  
    include_comments=True,  
    include_history=True  
)  
  
for bundle in exports:  
    task = bundle.task  
    print(f"Task: {task.title}")  
    print(f"Comments: {len(bundle.comments)}")  
    print(f"History entries: {len(bundle.history)}")  
  
    # Send to Jira, Linear, etc.  
    await external_system.sync_task(  
        title=task.title,  
        status=task.status,  
        comments=[c.content for c in bundle.comments]  
    )
```

Result: Each bundle contains:

List Findings

Use Case: Get critical security findings

Python:

```
from cerebro_sdk import FindingService

findings_svc = FindingService(db)

findings = await findings_svc.list_findings(
    org_id=org_id,
    severity="critical", # or "high", "medium", "low"
    status="open",       # or "fixed", "accepted_risk"
    limit=100
)

for f in findings:
    print(f"{f.finding_id}: {f.title}")
    print(f"  Severity: {f.severity}")
    print(f"  Provider: {f.provider}")
    print(f"  First seen: {f.first_seen}")
    print(f"  Evidence: {f.evidence}")
```

TypeScript:

```
const findings = await sdk.findings.list({
  orgId: "org-id",
```

Paginate Through Findings

Use Case: Process thousands of findings

Python:

```
from cerebro_sdk.pagination import PageRequest

page_request = PageRequest(cursor=None, limit=100)

while True:
    page = await findings_svc.list_findings_page(
        org_id=org_id,
        severity="critical",
        page=page_request
    )

    for finding in page.items:
        await process_finding(finding)

    if not page.next_cursor:
        break  # No more results

    page_request.cursor = page.next_cursor
```

TypeScript - With Helper:

```
import { iterateCursor } from "@writerai/cerebro-sdk";
```

Close a Finding

Use Case: Mark finding as resolved

Python:

```
success = await findings_svc.close_finding(  
    finding_id=finding.finding_id,  
    status="fixed"   # or "accepted_risk"  
)
```

```
if success:  
    print("Finding closed successfully")
```

Generate Findings for Org:

```
result = await findings_svc.generate_for_org(  
    org_id=org_id,  
    provider="aws",   # Optional: specific provider  
    resource_types=["ec2_instance", "s3_bucket"] # Optional  
)
```

```
print(f"Generated {result.findings_created} new findings")  
print(f"Updated {result.findings_updated} existing findings")  
print(f"Errors: {len(result.errors)}")
```

Create Tool Invocation

Use Case: Track agent tool execution

Python:

```
from cerebro_sdk import AgentToolingManager
from cerebro.agents.models import ToolInvocationStatus

tooling = AgentToolingManager(db)

invocation = await tooling.create_invocation(
    session_id=session.session_id,
    tool_name="timeline.generate",
    input_data={"scope": "finding", "finding_id": "finding-123"},
    status=ToolInvocationStatus.RUNNING
)

print(f"Invocation ID: {invocation.invocation_id}")
```

Update with result:

```
updated = await tooling.update_invocation_result(
    invocation_id=invocation.invocation_id,
    status=ToolInvocationStatus.SUCCESS,
    output_data={"timeline": [...], "event_count": 42},
    cel_result=True # CEL policy passed
)
```

List Tool Invocations

Use Case: Audit what tools were called

Python:

```
invocations = await tooling.list_invocations(  
    org_id=org_id,  
    session_id=session_id, # Optional  
    tool_name="timeline.generate", # Optional  
    status=ToolInvocationStatus.SUCCESS, # Optional  
    text_query="critical" # Optional: search inputs/outputs  
)
```

```
for inv in invocations:  
    print(f"{inv.tool_name} at {inv.created_at}")  
    print(f"  Status: {inv.status}")  
    print(f"  Duration: {inv.duration_seconds}s")  
    print(f"  Input: {inv.input_data}")  
    print(f"  Output: {inv.output_data}")
```

Get Summary Statistics:

```
summaries = await tooling.summarize_invocations(  
    org_id=org_id,  
    status=ToolInvocationStatus.SUCCESS  
)
```

List and Register Vendors

Use Case: Vendor risk management

TypeScript:

// List vendors with filters

```
const vendors = await sdk.securityCenter.listVendors("org-id", {
  riskLevel: "high",
  category: "security",
  limit: 50
});
```

```
vendors.forEach(v => {
  console.log(`${v.name}: ${v.riskLevel} risk`);
  console.log(`  Last review: ${v.lastReviewDate}`);
  console.log(`  Criticality: ${v.businessCriticality}`);
});
```

// Register new vendor

```
await sdk.securityCenter.registerVendor("org-id", {
  name: "Acme Security Platform",
  websiteUrl: "https://acme.example.com",
  category: "security",
  businessCriticality: "high",
  primaryContact: {
    name: "John Doe",
```

Assess Vendor Health

Use Case: Compliance review status

TypeScript:

```
import { assessVendorHealth } from "@writerai/cerebro-sdk";

const vendors = await sdk.securityCenter.listVendors("org-id");

const assessments = vendors.map(vendor =>
  assessVendorHealth(vendor)
);

// Filter for issues
const needsReview = assessments.filter(a =>
  a.reviewStatus === "overdue"
);

const expiringSoon = assessments.filter(a =>
  a.daysUntilExpiration !== null &&
  a.daysUntilExpiration < 30
);

console.log(`Overdue reviews: ${needsReview.length}`);
console.log(`Expiring soon: ${expiringSoon.length}`);
```


Iterate Through All Vendors

Use Case: Process entire vendor portfolio

TypeScript:

```
// Automatically handles pagination
const allVendors = await sdk.securityCenter.iterateVendors(
  "org-id",
  { limit: 100 }
);

console.log(`Total vendors: ${allVendors.length}`);

// Or stream with async iteration
for await (const vendor of sdk.securityCenter.iterateVendorsStream(
  "org-id"
)) {
  await processVendor(vendor);
}
```

Get Portfolio Summary:

```
import { summarizeVendorPortfolio } from "@writerai/cerebro-sdk";

const summary = summarizeVendorPortfolio(allVendors);

console.log(`By risk: ${JSON.stringify(summary.byRiskLevel)}`);
```

Get Runtime Health

Use Case: Monitor agent performance

Python:

```
from cerebro_sdk import RuntimeHealthClient

health = RuntimeHealthClient(db)

metrics = await health.get_runtime_health(
    org_id=org_id,
    since_hours=24  # Last 24 hours
)

print(f"Total events: {metrics.total_events}")
print(f"Errors: {metrics.error_count}")
print(f"Warnings: {metrics.warning_count}")
print(f"Avg response time: {metrics.avg_response_time_ms}ms")

# Event breakdown by type
for event_type, count in metrics.event_types.items():
    print(f"  {event_type}: {count}")
```

Integration Coverage:

```
coverage = await health.get_integration_coverage(org_id=org_id)
```

Pattern: Automated Finding Triage

Use Case: Auto-create agent sessions for critical findings

```
from cerebro_sdk import FindingService, AgentManager

async def auto_triage_critical(org_id):
    async with async_session_factory() as db:
        findings_svc = FindingService(db)
        agents = AgentManager(db)

        # Get new critical findings
        findings = await findings_svc.list_findings(
            org_id=org_id,
            severity="critical",
            status="open",
            limit=10
        )

        for finding in findings:
            # Create agent session
            session = await agents.create_session(
                org_id=org_id,
                agent_type="security_analyst",
                created_by="automation@writer.com",
                context={"finding_ids": [str(finding.finding_id)]},
                title=f"Triage: {finding.title}"
```

Pattern: Review Queue Dashboard

```
from cerebro_sdk import AgentReviewManager

async def get_queue_metrics(org_id):
    async with async_session_factory() as db:
        review = AgentReviewManager(db)

        # Get pending tasks
        pending, _ = await review.list_tasks(
            org_id=org_id,
            status="pending"
        )

        # Get SLA at-risk
        at_risk = await review.list_tasks_sla_at_risk(
            org_id=org_id,
            threshold_hours=24
        )

        # Get summary
        summary = await review.get_queue_summary(org_id)

    return {
        "pending_count": len(pending),
        "at_risk_count": len(at_risk),
```

Pattern: Compliance Report

```
import { CerebroSDK, assessVendorHealth,
        summarizeVendorPortfolio } from "@writerai/cerebro-sdk";

async function generateComplianceReport(orgId: string) {
  const sdk = new CerebroSDK({...});

  // Get all vendors
  const vendors = await sdk.securityCenter.iterateVendors(orgId);

  // Assess health
  const assessments = vendors.map(assessVendorHealth);

  // Calculate metrics
  const overdue = assessments.filter(a =>
    a.reviewStatus === "overdue"
  );
  const expiring = assessments.filter(a =>
    a.daysUntilExpiration && a.daysUntilExpiration < 30
  );

  const portfolio = summarizeVendorPortfolio(vendors);

  return {
    totalVendors: vendors.length,
```

Pattern: Tool Execution Audit

```
from cerebro_sdk import AgentToolingManager
from cerebro.agents.models import ToolInvocationStatus

async def audit_tool_usage(org_id, days=7):
    async with async_session_factory() as db:
        tooling = AgentToolingManager(db)

        # Get all tool calls
        invocations = await tooling.list_invocations(
            org_id=org_id,
            limit=1000
        )

        # Group by tool
        by_tool = {}
        for inv in invocations:
            if inv.tool_name not in by_tool:
                by_tool[inv.tool_name] = {
                    "total": 0, "success": 0, "failed": 0,
                    "total_duration": 0
                }

            by_tool[inv.tool_name]["total"] += 1
            if inv.status == ToolInvocationStatus.SUCCESS:
```

Python SDK Method Cheat Sheet

Module	Key Methods
AgentManager	<code>create_session()</code> , <code>add_message()</code> , <code>list_sessions()</code> <code>get_memory_stats()</code> , <code>link_findings()</code> , <code>sessions_for_finding()</code>
AgentReviewManager	<code>list_tasks()</code> , <code>bulk_update_tasks()</code> , <code>add_comment()</code> <code>get_queue_summary()</code> , <code>export_tasks()</code> , <code>list_tasks_sla_at_risk()</code>
FindingService	<code>list_findings()</code> , <code>list_findings_page()</code> , <code>close_finding()</code> <code>generate_for_org()</code> , <code>get_finding()</code>
AgentToolingManager	<code>create_invocation()</code> , <code>update_invocation_result()</code> <code>list_invocations()</code> , <code>summarize_invocations()</code>
RuntimeHealthClient	<code>get_runtime_health()</code> , <code>get_integration_coverage()</code>
AuthSession	<code>login()</code> , <code>verify()</code> , <code>refresh()</code>

TypeScript SDK Method Cheat Sheet

Client	Key Methods
<code>sdk.agents</code>	<code>createSession()</code> , <code>listSessions()</code> , <code>getSession()</code> <code>streamSession()</code> , <code>listSessionMessages()</code>
<code>sdk.agents (review)</code>	<code>listReviewTasks()</code> , <code>bulkUpdateReviewTasks()</code> <code>getReviewQueueSummary()</code> , <code>listReviewTasksSlaAtRisk()</code>
<code>sdk.findings</code>	<code>list()</code> , <code>get()</code> , <code>close()</code>
<code>sdk.securityCenter</code>	<code>listVendors()</code> , <code>registerVendor()</code> , <code>listCustomers()</code> <code>iterateVendors()</code> , <code>iterateCustomers()</code>
<code>sdk.analytics</code>	<code>getRuntimeHealth()</code> , <code>getIntegrationCoverage()</code>

Helper Functions:

- ▶ `iterateCursor()`, `collectCursor()` - pagination
- ▶ `collectAgentStream()` - streaming
- ▶ `assessVendorHealth()`, `assessCustomerHealth()` - analytics
- ▶ `summarizeVendorPortfolio()`, `summarizeCustomerPortfolio()`

Common Patterns Quick Reference

Pattern	Key Methods
Auto-triage findings	<code>list_findings()</code> + <code>create_session()</code> + <code>add_message()</code>
Review queue dashboard	<code>list_tasks()</code> + <code>list_tasks_sla_at_risk()</code> + <code>get_queue_summary()</code>
Compliance report	<code>iterateVendors()</code> + <code>assessVendorHealth()</code> + <code>summarizeVendorPortfolio()</code>
Tool usage audit	<code>list_invocations()</code> + <code>summarize_invocations()</code>
Session investigation	<code>get_session()</code> + <code>list_messages()</code> + <code>get_memory_stats()</code>
Paginate large datasets	<code>list*_page()</code> + <code>cursor</code> or <code>iterateCursor()</code> helper

Where To Get Help

Documentation:

- ▶ `docs/sdk/README.md` - SDK overview
- ▶ `docs/sdk/python.md` - Python guide
- ▶ `docs/sdk/typescript.md` - TypeScript guide
- ▶ `docs/sdk/auth-and-agents.md` - Agent workflows

Code Examples:

- ▶ `examples/` - Real-world scenarios
- ▶ `tests/unit/sdk/` - Python unit tests (great examples!)
- ▶ `sdk/ts/test/` - TypeScript test suite

Get Support:

- ▶ Slack: `#cerebro-sdk`
- ▶ Office hours: Tuesdays 2-3pm
- ▶ Email: `cerebro-sdk@writer.com`

Next Steps

Right Now:

1. Pick one use case from this deck
2. Copy the code example
3. Run it in your project
4. Modify for your needs

This Week:

- ▶ Build a simple integration (list sessions, get findings, etc.)
- ▶ Try pagination with real data
- ▶ Add error handling

Questions?

- ▶ Drop them in #cerebro-sdk
- ▶ Or ask at office hours Tuesday 2pm